



Prerequisites (Detailed)

Verify Hetzner account: Log in at hetzner.com/cloud; top up €20 credit if needed (India card works).^[1]

Create VPS:

- Console > New Server > CX22 (AMD 2vCPU/4GB ECC RAM/40GB NVMe, €3.79/mo excl VAT).
- Location: Falkenstein/Nuremberg (low India ping ~80ms); Mumbai peering via Cloudflare.
- Add SSH key: Generate `ssh-keygen -t ed25519 -C "your@email.com"` locally, copy `~/.ssh/id_ed25519.pub` to Hetzner.
- Networking: Enable IPv4 public IP; create firewall "coolify" (TCP 22,80,443,8000,3000,5432). SSH: `ssh root@IP -i ~/.ssh/id_ed25519` (accept host key).^{[2] [3]}

Domain: In Cloudflare, add A record @ IP, *. IP (proxied orange cloud for DDoS/CDN).^[4]

Repos ready:

- FastAPI: `requirements.txt` with `fastapi uvicorn redis supabase bge-large-en-v1.5 psycopg2`.
- Next.js: Standard `next build && next start`.
- GitHub private/public; generate token if private.^[5]

Step 1: VPS System Prep (Detailed)

SSH in:

```
apt update && apt full-upgrade -y && apt autoremove -y
apt install curl wget htop ufw -y
ufw --force enable
ufw allow 22/tcp # SSH
ufw allow 80/tcp 443/tcp 8000/tcp 3000/tcp 5432/tcp # Coolify, apps
ufw status verbose
reboot
```

Post-reboot SSH back; set hostname `hostnamectl set-hostname coolify-server`.^{[6] [2]}

Step 2: Coolify Installation (Detailed)

```
curl -fsSL https://cdn.coollabs.io/coolify/install.sh | bash
```

- Script installs Docker 27+, Docker Compose, Traefik, sets up systemd.
- Output: "Coolify installed! Access at <http://IP:8000>"
Browser: <http://IP:8000> > Set admin password > Skip team for solo.
Settings > SSH Keys: Paste your private key (cat ~/.ssh/id_ed25519).^[3] ^[6]

Step 3: Add Server & Domain (Detailed)

Servers tab > "Add Server":

- Name: "Hetzner-Primary", IP: YOUR_VPS_IP, Credential: Select SSH key.
- Validate: Coolify SSHes, installs Docker Swarm agent (~2min).
Domains > "Add Domain":
- Domain: yourdomain.com
- SSL: Auto (Let's Encrypt via Traefik).
- Test: Coolify pings; green check.^[3] ^[4]

Step 4: Deploy Self-Hosted Supabase (Detailed)

Applications > New > "Deploy from Marketplace" > Search "Supabase" (official ajinidop/supabase).

- Server: Hetzner-Primary.
- Project: New "supabase".
- Ports: Auto-map (Studio 8000→<https://studio.yourdomain.com>).
- Env vars (edit post-deploy): POSTGRES_PASSWORD=strongpass, JWT_SECRET=gen_jwt_secret.
Deploy (~5min build/pull).
Access: <https://studio.yourdomain.com> > Sign up, create project "prod".
Integrate Redis: Edit compose in Coolify > Services > supabase > Edit docker-compose.yml:

```
services:
  db:
    ...
  redis: # Add
    image: redis:7-alpine
    volumes: ['./volumes/redis:/data']
    restart: unless-stopped
```

Redeploy.^[7] ^[8]

Step 5: Deploy Redis Standalone (Detailed)

If separate: Applications > New > "Docker Compose".

- Name: "Redis", Server: Primary.
- Upload/Paste docker-compose.yml:

```
version: '3.8'
services:
  redis:
    image: redis:7-alpine
    restart: unless-stopped
    ports:
      - '6379:6379'
    volumes:
      - './volumes/redis:/data'
    command: redis-server --appendonly yes --maxmemory 1gb --maxmemory-policy allkeys-lru
```

- Deploy; internal connect: redis.internal:6379 (Coolify DNS).^[6]

Step 6: Deploy FastAPI (Detailed)

Applications > New > "Git Repository".

- Git: github.com/YOUR/fastapi-repo (token if private).
- Branch: main.
- Build Pack: Docker (auto Dockerfile detect).
Dockerfile (in repo root):

```
FROM python:3.12-slim AS builder
WORKDIR /app
COPY pyproject.toml poetry.lock ./
RUN pip install poetry && poetry export -f requirements.txt --output requirements.txt
COPY . .

FROM python:3.12-slim
WORKDIR /app
COPY --from=builder /app/requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8000
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000", "--workers", "2"]
```

Env vars:

- REDIS_URL=redis://redis.internal:6379
- DATABASE_URL=postgresql://postgres:strongpass@supabase-db.internal:5432/postgres
- Ports: 8001 → <https://api.yourdomain.com>
Deploy (~3min); test curl <https://api.yourdomain.com/health>.^[9] ^[5]

Step 7: Deploy Next.js (Detailed)

Applications > New > "Git Repository".

- Git: github.com/YOUR/nextjs-repo.
- Build Pack: Next.js (auto).
- Install Cmd: `npm ci --include=dev`
- Build Cmd: `npm run build`
- Start Cmd: `npm start`
Env:
- NEXT_PUBLIC_API_URL=`https://api.yourdomain.com`
- Ports: 3000 → <https://app.yourdomain.com>
Deploy (~4min build).
Optimize: Add `next.config.js` output: 'standalone' for prod; static to Cloudflare Pages if pure static. ^[10] ^[4]

Verification & Next

- Logs: Coolify > App > Logs (tail realtime).
- Health: Browser all subdomains; `docker ps` on VPS.
- Git push: Auto-deploy in <1min.
- Flutter: Update API base URL to `https://api.yourdomain.com`.
Add Qdrant/PostHog similarly (one-click Docker Compose). Backup: Coolify > Backups > Schedule daily. ^[11] ^[12] ^[13]

✱

1. <https://cloudpricecheck.com/hetzner/cloud-servers-pricing>
2. https://www.youtube.com/watch?v=rtYhj9v3k_s
3. <https://coolify.io/docs/knowledge-base/how-to/webstudio-with-hetzner>
4. <https://shipixen.com/boilerplate-documentation/coolify-cloud-nextjs-deploy-integration>
5. <https://fastapi.tiangolo.com/deployment/docker/>
6. <https://moreslot.com/coolify-docker-guide/>
7. <https://dev.to/musayazlik/harnessing-the-power-of-self-hosted-supabase-on-coolify-a-complete-guide-to-server-setup-and-oauth-34c5>
8. <https://www.youtube.com/watch?v=NFjvle2VfZQ>
9. <https://railway.com/deploy/fastapi-clerk-supabase-redis>
10. <https://dev.to/mandrasch/deploy-sveltekit-with-ssr-on-coolify-hetzner-vps-24c5>
11. <https://www.youtube.com/watch?v=gLf12xm5tDg>
12. <https://posthog.com/docs/self-host>

13. <https://www.perplexity.ai/search/Of85cef1-5e89-4f19-a05d-62743b221052>